German University in Cairo
Faculty of Media Engineering and Technology
Dr. Aya Salama

CSEN 602 Operating Systems, Spring 2026
Project Description
Due Date: 18/4/2026 at 11:59pm

Project Objective

The best way for you to understand the concepts of an Operating System is to build an operating system and then to experiment with it to see how the OS manages resources and processes. In this project, you are asked to build a simulation of an operating system. The main focus of the project will be on building a correct architecture that simulates a real operating system. **You will be graded on both a correct implementation and a correct architecture.**

Description

In this project, you are asked to implement a basic interpreter. You have a text file that represents a program. When you read that text file and start executing it, it becomes a process. **You are provided with 3 program files each rep- resenting a program. You are asked to create an interpreter that reads the txt files and executes their code. You are asked to implement a memory and save the processes in it. You are also asked to implement mutexes that ensure mutual exclusion over the critical resources. And finally, you are also asked to implement a scheduler that schedules the processes that we have in our system.**

System Calls

System calls are the process's way of requesting a service from the OS. In order for a process to be able to use any of the available hardware, it makes a request, system call, to the operating system.

Types of system calls required:

1. Read the data of any file from the disk.
2. Write text output to a file in the disk.
3. Print data on the screen.

4. Take text input from the user.
5. Reading data from memory.
6. Writing data to memory.

Memory

One of the steps the OS does in order to create a new process is allocating a space for it in the main memory. The OS is responsible for managing the memory and its allocation.

The memory is of a fixed size. It is made up of 40 memory words. The memory is large enough to hold the un-parsed lines of code, variables and PCB for any of the processes. The memory is divided into memory words, each word can store 1 variable and it's corresponding data. For simplicity, feel free to specify a naming convention for the variable names associated with the lines of code and elements of the PCB. Processes should not access any data outside their allocated memory block.

A process should only be created at it's arrival time. A process is considered created when it's program file is read into lines and it gets assigned a part of the memory for instructions, variables and it's PCB. Assume that each process needs enough space for 3 variables.

Feel free to separate the lines of code, variables and PCB within the memory if needed as long as they fall within the same data structure meant to represent the memory.

Your code needs to be able to handle the case where the memory is not large enough to run all the processes. When a new process is created, the system checks if there is enough space, if not the system will unload one of the existing processes and store it's data on the disk. Feel free to assume the format for the memory when stored on the disk. However, while unloaded, the process remains in the scheduler, thus when it is time for the unloaded process to run again, the process memory is swapped back into the memory from the disk. As a result, you need to program a way to read and write an existing process's data to/from the disk as well as manage the swapping between the process's while protecting each process's memory.

Process Control Block

A process control block is a data structure used by computer operating systems to store all the information about a process.In order to schedule your processes, you

will need to keep a PCB for every process. The PCB should contain the following information:

1. Process ID (The process is given an ID when is being created)
2. Process State
3. Program Counter
4. Memory Boundaries

Programs

We have 3 main Programs:

Program 1: Given 2 numbers, the program prints the numbers between the 2 given numbers on the screen.
Program 2: Given a filename and data, the program writes the data to the file. Assume that the file doesn't exist and should always be created.
Program 3: Given a filename, the program prints the contents of the file on the screen.

Program Syntax

For the programs, the following syntax is used:

- print: to print the output on the screen. Example: print x
- assign: to initialize a new variable and assign a value to it. Example: assign x y, where x is the variable and y is the value assigned. The value could be an integer number, or a string. If y is input, it first prints to the screen **"Please enter a value"**, then the value is taken as an input from the user.
- writeFile: to write data to a file. Example: writeFile x y, where x is the filename and y is the data.
- readFile: to read data from a file. Example: readFile x, where x is the filename
- printFromTo: to print all numbers between 2 numbers. Example: printFromTo x y, where x is the first number, and y is the second number.

- semWait: to acquire a resource. Example: semWait x, where x is the resource name. For more details refer to section **Mutual Exclusion**
- semSignal: to release a resource. Example: semSignal x, where x is the resource name. For more details refer to section **Mutual Exclusion**

Mutual Exclusion

A mutex is a directive provided by the OS used to control access to a shared resource between processes in a concurrent system such as a multi-programming operating system by using two atomic operations, semwait and semsignal. Mutexes are used to ensure mutual exclusion over the critical section.

You are required to implement 3 mutexes, one for each resource we have:

1. Accessing a file, to read or to write.
2. Taking user input
3. Outputting on the screen.

Whenever a mutex is used, either a semWait or semSignal instruction is followed by the name of the resource, **userInput, userOutput or file**. For an illustration, to print on the screen:

1. **semWait userOutput:** any process calls it whenever it wants to print something on the screen to acquire the key of the resource.
2. **semSignal userInput:** any process calls it whenever it finishes printing to release the key of the resource.

Note: ONLY ONE process is allowed to use the resource at a time. If a process requests the use of a resource while it is being used by another process, it should be blocked and added to the blocked queue of this resource and the general blocked queue.

Scheduler

A scheduler is responsible for scheduling between the processes in the Ready Queue. It ensures that all processes get a chance to execute. A scheduling algorithm is an algorithm that chooses the process that gets to execute. As mentioned in the lecture, there are many different scheduling algorithms to schedule processes.

In this project, you are required to implement multiple scheduling algorithms: **Highest Response Ratio Next (HRRN)** and **Round Robin (RR)** as part of the **project requirements**.
The **Multi-Level Feedback Queue (MLFQ)** scheduling technique will be **graded as part of the assignment grade(5%) rather than the project grade**.

1) **Highest Response Ratio Next (HRRN)** is a non-preemptive scheduling algorithm that selects the process with the highest response ratio. The response ratio is calculated using the formula: Response Ratio = (Waiting Time + Burst Time) / Burst Time. At every scheduling decision, the process with the highest response ratio in the Ready Queue is selected to run.
2) **Round Robin (RR)** is a preemptive scheduling algorithm where each process is assigned a fixed time slice. For this project, each process executes 2 instructions per time slice. If a process does not finish within its time slice, it is moved to the end of the Ready Queue, and the next process is scheduled.
3) **Multi-Level Feedback Queue (MLFQ)** is a scheduling algorithm that uses multiple priority queues. The system consists of 4 priority queues. Processes start in the highest-priority queue. The quantum length for the highest level is 1 time unit, and the quantum of each queue follows the formula 2^i, where i is the queue level index starting from 0. If a process uses its entire quantum, it is moved to a lower-priority queue. The scheduler always selects a process from the highest-priority non-empty queue. The last queue is scheduled using Round Robin.

- **Note:** The implementation of **MLFQ will be graded as part of the assignment grade (5%), not the project grade.**

Processes arrive in this order:

- Process 1 arrives at time 0
- Process 2 arrives at time 1
- Process 3 arrives at time 4
-

Queues

- Ready Queue: For the processes currently waiting to be chosen to execute on the processor
- Blocked Queue: For the processes currently waiting for resources to be avail- able

Output

For this project, your Simulated OS should be able to read the provided programs and execute them. You should make sure to have the following outputs read to show for the evaluation:

- Queues should be printed after every scheduling event, i.e. when a process is chosen, blocked, or finished.
- Which process is currently executing.
- The instruction that is currently executing
- Time slice is subject to change, i.e. you might be asked to change it to x instructions per time slice.
- Order in which the processes are scheduled are subject to change.
- The timings in which processes arrive are subject to change
- The memory shown every clock cycle in a human readable format.
- The ID of any process whenever it is swapped in or out of disk.
- The format of the memory stored on Disk.

Please make sure that the output is readable, and presentable.

Graphical User Interface (GUI)

The requirement of a Graphical User Interface (GUI) depends on the programming language used for the project:

- If the project is implemented using **Java**, the **GUI is mandatory** and will be considered part of the project evaluation.
- If the project is implemented using **C**, the **GUI is optional and considered a bonus component**.

The GUI aims to provide a visual representation of the system's behavior and make the simulation easier to understand and interact with. Through the GUI, users can observe how processes move between states and how system resources are managed in real time.

The GUI may include, but is not limited to, the following features:

- Visual display of the Ready and Blocked Queues
- Display of the currently running process
- Visualization of memory contents and disk swapping
- Step-by-step clock cycle execution view
- Clear indication of process state changes (Ready, Running, Blocked, Finished)
- Buttons or controls to start, pause, or step through execution.

Work Distribution

- Code Parser/Interpeter
- System Calls
- Mutexes
- Scheduler
- Memory

Project Deliverable and Submission

You will work in teams of 4-5 members. The programming language that you will use is **JAVA or C**. The project should be submitted as ONE zip folder containing the java files you created. Please make sure to name your folder as follows, Team_number (ex. Team_00). Late submissions will not be accepted.